Rupi — Dokumen Kebutuhan Sistem

Oct 31, 2026 · @Surya Suharna

Ringkasan produk

Rupi adalah asisten pencatatan keuangan pribadi untuk mahasiswa yang dipakai sepenuhnya lewat chat dengan bahasa sehari-hari. Pengguna cukup mengetik "beli bakso 15rb" dan Rupi mencatat, mengkategorikan, serta melaporkannya.

Masalah. Mahasiswa jarang konsisten mencatat pengeluaran karena aplikasi keuangan menuntut membuka form, memilih kategori, dan mengisi tanggal. Akibatnya uang "bocor" tanpa jejak dan baru terasa di akhir bulan.

Tujuan produk:

- Mencatat transaksi dalam waktu kurang dari 5 detik lewat satu pesan chat.
- Memahami bahasa santai Indonesia: singkatan nominal, slang, dan waktu relatif.
- Memberi gambaran pengeluaran dan peringatan budget tanpa pengguna perlu membuka dashboard.

Tujuan belajar (proyek ini juga sarana belajar):

- Merancang microservices dengan kontrak gRPC/protobuf, termasuk streaming.
- Men-deploy dan mengoperasikan sistem di Kubernetes: scaling, CronJob, StatefulSet, Secret, observability.
- Mengintegrasikan LLM secara aman sebagai komponen yang menghasilkan data terstruktur, bukan menjalankan aksi.

Target pengguna

Pengguna utama adalah mahasiswa S1 berusia 18–24 tahun yang mengelola uang saku bulanan sendiri dan aktif memakai Telegram atau WhatsApp.

Persona	Situasi	Kebutuhan utama dari Rupi
Dina, mahasiswa rantau	Tinggal di kos, uang saku Rp1,5 juta/bulan dari orang tua	Tahu sisa uang per minggu, tidak kehabisan sebelum transfer berikutnya
Raka, mahasiswa sambil kerja	Penghasilan freelance tidak menentu	Mencatat pemasukan dan pengeluaran, melihat arus kas bulanan

Persona	Situasi	Kebutuhan utama dari Rupi
Sari, bendahara kos	Mengurus patungan listrik, wifi, galon	Dompet bersama dan pengingat tagihan ke anggota (fase lanjutan)

Admin sistem (pengembang) juga menjadi pengguna internal: memantau kesehatan service, biaya LLM, dan akurasi parser.

Ruang lingkup

MVP fokus pada satu kanal (Telegram) dan satu alur inti: catat, koreksi, tanya ringkasan. Fitur lain menyusul setelah alur inti stabil.

Kelompok	Fitur	Fase
Pencatatan	Catat pengeluaran/pemasukan lewat bahasa natural, banyak transaksi per pesan	MVP
Pencatatan	Koreksi dan hapus transaksi lewat chat ("yang bakso tadi 18rb", "batalin yang terakhir")	MVP
Pencatatan	Kategorisasi otomatis + kategori kustom	MVP
Laporan	Tanya ringkasan dengan bahasa natural ("bulan ini habis berapa buat makan?")	MVP
Laporan	Laporan mingguan dan bulanan otomatis	Lanjutan 1
Budget	Budget per kategori per periode + peringatan proaktif	Lanjutan 1
Kanal	Web chat dan dashboard grafik	Lanjutan 2
Kanal	WhatsApp	Lanjutan 2
Pencatatan	Foto struk → OCR → transaksi per item	Lanjutan 2
Sosial	Dompet bersama (patungan kos, kas kepanitiaan)	Lanjutan 3
Data	Ekspor CSV/Excel	Lanjutan 3

Di luar lingkup: integrasi langsung ke rekening bank atau e-wallet, transaksi pembayaran sungguhan, saran investasi, dan aplikasi mobile native.

Kebutuhan fungsional

Setiap kebutuhan diberi kode FR agar bisa dirujuk dari test case dan issue. Prioritas: **P0** wajib di MVP, **P1** fase lanjutan 1, **P2** sesudahnya.

Akun dan onboarding

Kode	Kebutuhan	Prioritas
FR-01	Pengguna terdaftar otomatis saat pertama kali mengirim /start ke bot, berdasarkan ID Telegram	P0
FR-02	Onboarding singkat: nama panggilan, zona waktu (default WIB), saldo awal opsional	P0
FR-03	Pengguna dapat menghapus seluruh datanya dengan perintah /hapusakun dan konfirmasi	P0
FR-04	Pengguna dapat menghubungkan akun web dengan akun Telegram lewat kode sekali pakai	P2

Pencatatan transaksi

Kode	Kebutuhan	Prioritas
FR-10	Sistem mengubah pesan bahasa natural menjadi satu atau lebih transaksi (jenis, nominal, deskripsi, kategori, waktu)	P0
FR-11	Satu pesan dapat berisi beberapa transaksi ("bakso 15rb sama es teh 5rb")	P0
FR-12	Jika keyakinan parser di bawah ambang (default 0,7) atau ada data yang kurang, bot bertanya balik sebelum menyimpan	P0
FR-13	Setiap transaksi yang tersimpan dikonfirmasi dengan ringkasan dan tombol Ubah / Batalkan	P0
FR-14	Pengguna dapat mengoreksi transaksi dengan merujuknya secara natural ("yang bakso tadi harusnya 18rb")	P0
FR-15	Perintah "batalin yang terakhir" atau /undo menghapus transaksi terakhir dalam 24 jam	P0

Kode	Kebutuhan	Prioritas
FR-16	Pesan non-transaksi (sapaan, pertanyaan umum) dijawab singkat tanpa membuat transaksi	P0
FR-17	Pengguna dapat membuat kategori kustom dan aturan ("kalau ada kata 'kopken' masukkan ke Jajan")	P1
FR-18	Pengguna mengirim foto struk dan sistem mengusulkan transaksi per item untuk dikonfirmasi	P2

Laporan dan pertanyaan

Kode	Kebutuhan	Prioritas
FR-20	Pengguna bertanya dengan bahasa natural dan sistem menjawab dari data (total, per kategori, per periode, transaksi terbesar)	P0
FR-21	Perintah /ringkas menampilkan ringkasan hari ini, minggu ini, dan bulan ini	P0
FR-22	Laporan mingguan otomatis setiap Senin 08.00 dan laporan bulanan setiap tanggal 1	P1
FR-23	Perbandingan dengan periode sebelumnya ("bulan ini 20% lebih boros untuk makan")	P1
FR-24	Ekspor transaksi ke CSV untuk rentang tanggal tertentu	P2

Budget dan peringatan

Kode	Kebutuhan	Prioritas
FR-30	Pengguna menetapkan budget per kategori atau total, per minggu atau per bulan, lewat chat	P1
FR-31	Sistem mengirim peringatan saat pemakaian budget mencapai 80% dan 100%	P1
FR-32	Sistem memperkirakan sisa uang harian ("sisa Rp340rb untuk 12 hari, aman ~Rp28rb/hari")	P1

Kode	Kebutuhan	Prioritas
FR-33	Pengingat harian opsional jika pengguna belum mencatat apa pun hari itu	P1

Dompet bersama

Kode	Kebutuhan	Prioritas
FR-40	Pengguna membuat dompet bersama dan mengundang anggota lewat tautan	P2
FR-41	Transaksi di dompet bersama mencatat siapa yang membayar dan pembagiannya	P2
FR-42	Sistem menghitung siapa berutang ke siapa dan mengirim pengingat	P2

Pemahaman bahasa natural

Parser memakai dua lapis: aturan (regex + kamus) untuk pola umum, lalu LLM untuk kalimat yang tidak tertangani aturan. LLM hanya mengembalikan JSON terstruktur yang divalidasi; ia tidak pernah menulis ke database atau menyusun SQL.

Format nominal yang wajib dikenali

Tulisan pengguna	Nilai (Rp)
15000, 15.000, 15,000, Rp15.000	15.000
15rb, 15 rb, 15ribu, 15k	15.000
1,5jt, 1.5jt, 1,5 juta, 1500k	1.500.000
goceng	5.000
ceban	10.000
goban	50.000
cepek / gocap (konteks lama)	100 / 50 — minta konfirmasi
sejuta, setengah juta	1.000.000 / 500.000

Nominal disimpan sebagai bilangan bulat rupiah (`int64`), tidak pernah sebagai float.

Waktu relatif

Default waktu transaksi adalah saat pesan diterima. Ekspresi yang dikenali: "tadi pagi/siang/malam", "kemarin", "kemarin lusa", "senin kemarin", "tgl 3", "minggu lalu". Waktu diselesaikan terhadap zona waktu pengguna. Tanggal di masa depan ditolak kecuali eksplisit sebagai rencana (fase lanjutan).

Jenis transaksi

- **Pengeluaran** (default): beli, bayar, jajan, isi, makan.
- **Pemasukan**: dapet, gajian, transferan, dikasih, cair (beasiswa).
- **Transfer antar-dompet**: "pindahin 100rb dari BCA ke Gopay" (fase lanjutan).

Kategori bawaan

Makanan & Minuman, Transportasi, Kos & Utilitas, Pulsa & Internet, Kuliah (buku, fotokopi, print), Belanja, Hiburan, Kesehatan, Sosial & Hadiah, Lainnya. Pemasukan: Uang Saku, Gaji/Freelance, Beasiswa, Lainnya.

Contoh kalimat dan hasil yang diharapkan

Input	Hasil yang diharapkan
beli bakso 15rb	1 pengeluaran, Rp15.000, Makanan & Minuman, sekarang
bakso 15rb sama es teh goceng	2 pengeluaran: Rp15.000 dan Rp5.000, Makanan & Minuman
kemarin isi bensin 30rb	1 pengeluaran, Rp30.000, Transportasi, tanggal kemarin
dapet transferan ortu 1,5jt	1 pemasukan, Rp1.500.000, Uang Saku
print tugas 3500	1 pengeluaran, Rp3.500, Kuliah
makan siang	Tanya balik: "Berapa nominalnya?"
yang bakso tadi 18rb	Koreksi transaksi "bakso" terakhir hari ini menjadi Rp18.000
bulan ini habis berapa buat makan?	Query: total pengeluaran, kategori Makanan & Minuman, bulan berjalan
halo rupi	Sapaan, tanpa transaksi

Kumpulan contoh ini menjadi awal dataset evaluasi (lihat bagian Pengujian), dengan target minimal 300 kalimat sebelum rilis MVP.

Kebutuhan non-fungsional

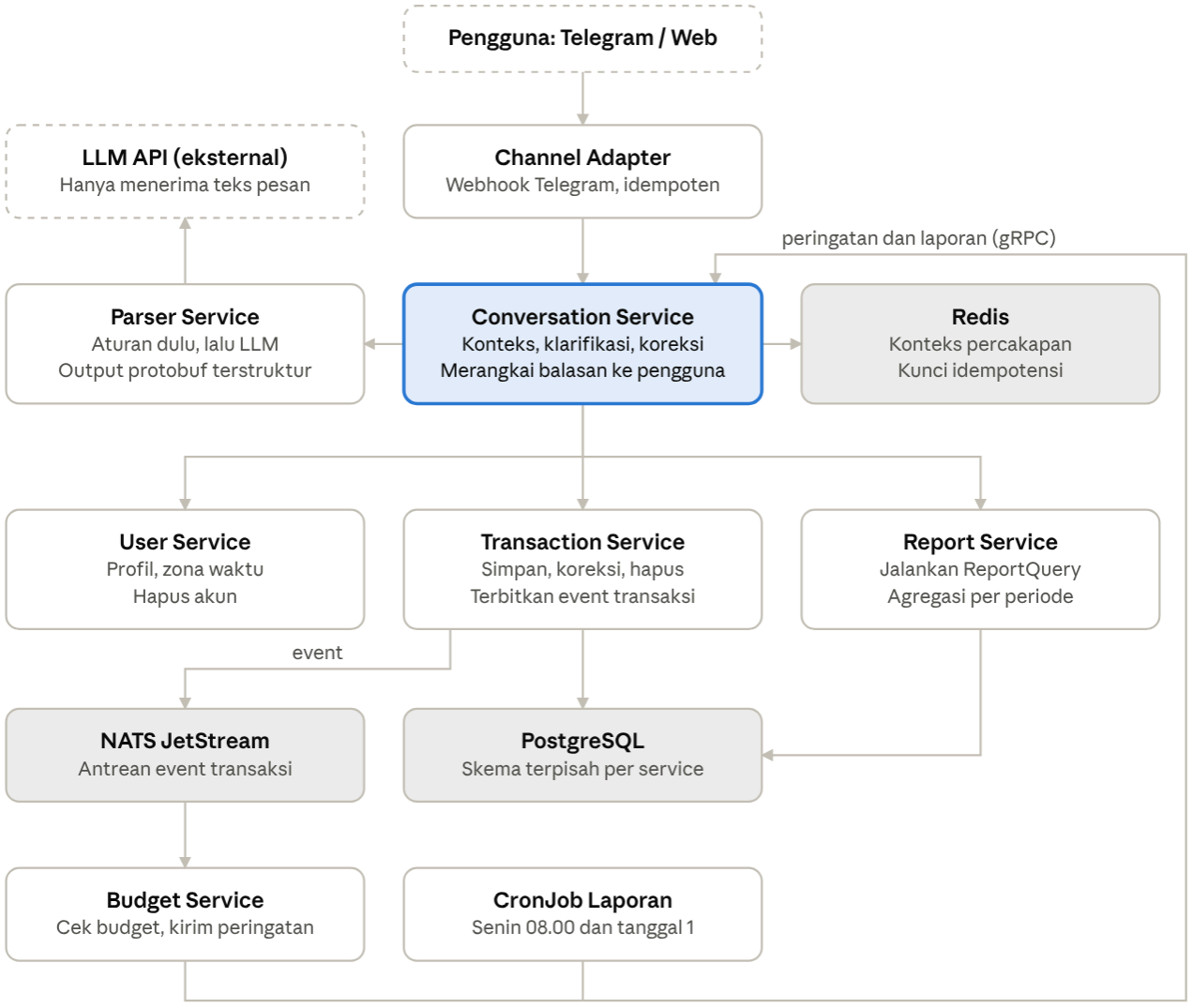
Target berikut berlaku untuk lingkungan uji dengan 1.000 pengguna simulasi; angka dapat disesuaikan setelah pengukuran pertama.

Kode	Aspek	Target
NFR-01	Latensi balasan, jalur aturan	p95 < 500 ms dari webhook diterima sampai balasan terkirim
NFR-02	Latensi balasan, jalur LLM	p95 < 3 detik; batas waktu panggilan LLM 5 detik lalu fallback ke tanya balik
NFR-03	Akurasi parser	≥ 95% transaksi terekstrak benar pada dataset evaluasi (nominal, jenis, tanggal); kategori ≥ 85%
NFR-04	Porsi jalur aturan	≥ 60% pesan selesai tanpa memanggil LLM, untuk menekan biaya
NFR-05	Ketersediaan	99,5% per bulan untuk alur pencatatan; tiap service minimal 2 replika
NFR-06	Skalabilitas	Tahan 50 pesan/detik dengan HPA; Parser Service skala 2–10 Pod berdasarkan CPU dan antrean
NFR-07	Idempotensi	Webhook yang terkirim ulang tidak pernah menghasilkan transaksi ganda
NFR-08	Konsistensi	Transaksi yang sudah dikonfirmasi tidak boleh hilang; tulis ke database sebelum membalas pengguna
NFR-09	Observability	Setiap request punya trace ID lintas service; metrik latensi, error, dan biaya LLM tersedia di Grafana
NFR-10	Zero-downtime deploy	Rolling update tanpa pesan yang hilang; graceful shutdown maks 30 detik
NFR-11	Biaya	Biaya LLM < Rp50 per pengguna aktif per bulan (diukur lewat token usage)
NFR-12	Kualitas kode	Cakupan unit test ≥ 70% pada Parser dan Transaction Service; lint + test di CI

Arsitektur sistem

Rupi terdiri dari delapan komponen aplikasi yang berkomunikasi lewat gRPC, dengan Conversation Service sebagai satu-satunya pengatur alur percakapan.

Conversation Service mengatur setiap pesan dari chat sampai data



arsitektur Rupi · 8 service, 3 penyimpanan data

Kotak bergaris putus-putus berada di luar sistem; kotak berlatar abu-abu adalah penyimpanan data. Semua panah antar-service adalah panggilan gRPC, kecuali jalur event lewat NATS.

Alur satu pesan pengeluaran:

1. Telegram mengirim webhook ke Channel Adapter, yang memeriksa ID pesan di Redis untuk mencegah duplikasi.
2. Channel Adapter memanggil `ConversationService.HandleMessage`.

3. Conversation Service mengambil konteks dari Redis lalu memanggil `ParserService.Parse`.
4. Parser mencoba aturan; jika gagal, memanggil LLM dan memvalidasi hasilnya.
5. Untuk hasil transaksi, Conversation Service memanggil `TransactionService.CreateTransactions`.
6. Transaction Service menyimpan ke PostgreSQL dan menerbitkan event `transaction.created` ke NATS.
7. Conversation Service mengirim balasan konfirmasi ke pengguna.
8. Secara asinkron, Budget Service membaca event dan mengirim peringatan bila ambang budget terlewati.

Kontrak gRPC

Semua kontrak berada di satu repositori `proto/` dengan paket berversi `rupi.<service>.v1`, dikelola dengan `buf` (lint + breaking-change check di CI). Perubahan yang tidak kompatibel wajib membuat versi `v2`.

Service	RPC	Jenis
ConversationService	`HandleMessage(IncomingMessage) → Reply`	Unary
ConversationService	`SubscribeOutbound(SubscribeRequest) → stream OutboundMessage`	Server streaming
ParserService	`Parse(ParseRequest) → ParseResult`	Unary, deadline 5 dtk
TransactionService	`CreateTransactions`, `UpdateTransaction`, `DeleteTransaction`, `ListTransactions`, `FindRecent`	Unary
ReportService	`RunQuery(ReportQuery) → ReportResult`	Unary
BudgetService	`SetBudget`, `GetBudgetStatus`	Unary
UserService	`GetOrCreateUser`, `UpdateProfile`, `DeleteUser`	Unary

Pesan inti:

```
syntax = "proto3";
package rupi.parser.v1;

import "google/protobuf/timestamp.proto";
```

```
enum TransactionType {
  TRANSACTION_TYPE_UNSPECIFIED = 0;
  TRANSACTION_TYPE_EXPENSE = 1;
  TRANSACTION_TYPE_INCOME = 2;
  TRANSACTION_TYPE_TRANSFER = 3;
}

message ParseRequest {
  string user_id = 1;
  string text = 2;
  string timezone = 3;                        // contoh: Asia/Jakarta
  google.protobuf.Timestamp received_at = 4;
  repeated RecentTransaction context = 5;    // untuk koreksi "yang bakso
tadi"
}

message ParsedTransaction {
  TransactionType type = 1;
  int64 amount = 2;                           // rupiah, bilangan bulat
  string description = 3;
  string category = 4;
  google.protobuf.Timestamp occurred_at = 5;
  float confidence = 6;
}

message ParseResult {
  oneof result {
    TransactionBatch transactions = 1;
    ReportQuery query = 2;
    Correction correction = 3;
    Clarification clarification = 4;
    SmallTalk small_talk = 5;
  }
  ParserSource source = 10;                   // RULE atau LLM, untuk metrik
}

message ReportQuery {
  Metric metric = 1;                          // SUM, COUNT, TOP_N, BREAKDOWN
  TransactionType type = 2;
  repeated string categories = 3;
  DateRange range = 4;
  GroupBy group_by = 5;                       // CATEGORY, DAY, WEEK, MONTH
  int32 limit = 6;
}
```

Aturan umum:

- Setiap panggilan membawa metadata `x-request-id` dan `x-user-id`; interceptor meneruskannya ke log dan trace.
- Kode error memakai status gRPC standar: `INVALID_ARGUMENT` untuk input tak valid, `NOT_FOUND`, `ALREADY_EXISTS` untuk pesan duplikat, `DEADLINE_EXCEEDED`, `UNAVAILABLE` (boleh di-retry).
- Antar-service memakai mTLS; tidak ada service selain Channel Adapter yang terbuka ke internet.
- Retry hanya untuk RPC idempoten, maksimal 3 kali dengan exponential backoff.

Model data

PostgreSQL menjadi sumber kebenaran; setiap service memiliki skema (schema) sendiri dan tidak membaca tabel milik service lain secara langsung. Redis dipakai untuk konteks percakapan dan kunci idempotensi.

Tabel (skema)	Kolom utama	Catatan
`users` (user)	id UUID, telegram_id BIGINT unik, display_name, timezone, created_at, deleted_at	Soft delete lalu purge 30 hari
`categories` (transaction)	id, user_id (null = bawaan), name, type, keywords TEXT[]	Kategori kustom per pengguna
`transactions` (transaction)	id UUID, user_id, wallet_id, type, amount BIGINT, description, category_id, occurred_at TIMESTAMPTZ, source_message_id, parser_source, created_at, updated_at, deleted_at	Indeks (user_id, occurred_at) dan (user_id, category_id, occurred_at)
`transaction_edits` (transaction)	id, transaction_id, field, old_value, new_value, edited_at	Jejak audit koreksi
`processed_messages` (conversation)	channel, external_message_id, processed_at	Unik (channel, external_message_id) untuk idempotensi

Tabel (skema)	Kolom utama	Catatan
`budgets` (budget)	id, user_id, category_id null, period WEEK/MONTH, amount BIGINT, alert_levels INT[]	
`budget_alerts` (budget)	budget_id, period_start, level, sent_at	Mencegah peringatan ganda
`wallets` (transaction)	id, name, owner_id, is_shared	Fase dompet bersama
`wallet_members` (transaction)	wallet_id, user_id, role	Fase dompet bersama
`parser_logs` (parser)	id, user_id hash, text_redacted, result JSON, source, latency_ms, tokens, created_at	Untuk evaluasi; disimpan 30 hari

Konteks percakapan di Redis: kunci `conv:<user_id>` berisi 10 transaksi terakhir dan pertanyaan klarifikasi yang sedang menunggu, TTL 30 menit.

Perubahan skema dikelola dengan migrasi berversi (misalnya `golang-migrate`) yang dijalankan sebagai Kubernetes Job sebelum deploy.

Infrastruktur Kubernetes

Pengembangan memakai cluster lokal kind; namespace `rupi-dev` dan `rupi-staging` dipisah, dan semua manifest dikemas sebagai Helm chart.

Komponen	Resource Kubernetes	Replika	Request / Limit	Catatan
Channel Adapter	Deployment + Service + Ingress	2	100m / 256Mi	Satu-satunya yang menerima trafik luar (webhook Telegram)
Conversation Service	Deployment + Service	2	100m / 256Mi	Stateless; konteks di Redis
Parser Service	Deployment + Service + HPA	2–10	250m / 512Mi	Skala berdasarkan CPU 70%
Transaction Service	Deployment + Service	2	100m / 256Mi	Pemilik data transaksi
Report Service	Deployment + Service	2	200m / 256Mi	Query agregasi

Komponen	Resource Kubernetes	Replika	Request / Limit	Catatan
Budget Service	Deployment + Service	2	100m / 128Mi	Konsumen event transaksi
User Service	Deployment + Service	2	100m / 128Mi	
Laporan berkala	CronJob	—	200m / 256Mi	Senin 08.00 dan tanggal 1, WIB
Migrasi skema	Job (Helm pre-upgrade hook)	—	100m / 128Mi	
PostgreSQL	StatefulSet + PVC 10Gi	1	500m / 1Gi	Atau operator CloudNativePG
Redis	StatefulSet + PVC 1Gi	1	100m / 256Mi	
NATS JetStream	StatefulSet	1–3	100m / 256Mi	Event `transaction.created/updated/deleted`
Observability	Prometheus, Grafana, Loki, OpenTelemetry Collector	—	—	Namespace `monitoring`

Ketentuan wajib untuk setiap Deployment:

- `readinessProbe` dan `livenessProbe` memakai gRPC health checking protocol (`grpc.health.v1`).
- `PodDisruptionBudget` dengan `minAvailable: 1` .
- Konfigurasi lewat ConfigMap; token bot, API key LLM, dan kredensial database lewat Secret (tidak pernah di image atau Git).
- `NetworkPolicy` default deny; hanya jalur panggilan yang tercantum di arsitektur yang diizinkan.
- Load balancing gRPC antar-service memakai headless Service + client-side round robin, atau service mesh Linkerd, karena koneksi HTTP/2 yang long-lived tidak tersebar merata oleh Service biasa.
- Image dibangun di CI, ditandai dengan SHA commit, dan tidak memakai tag `latest` .

Keamanan dan privasi

Data keuangan bersifat pribadi, jadi setiap akses dibatasi ke pemiliknya dan data yang dikirim ke LLM diminimalkan.

Kode	Kebutuhan
SEC-01	Webhook Telegram diverifikasi dengan `secret_token`; request tanpa token valid ditolak
SEC-02	Setiap query di Transaction dan Report Service wajib difilter `user_id` dari metadata terautentikasi, bukan dari isi pesan
SEC-03	Komunikasi antar-service memakai mTLS (sertifikat dari cert-manager atau Linkerd)
SEC-04	Yang dikirim ke LLM hanya teks pesan dan daftar kategori; tidak ada nama, ID Telegram, atau riwayat lengkap
SEC-05	Output LLM divalidasi terhadap skema; nominal di luar Rp1 sampai Rp1 miliar atau tanggal tak wajar ditolak
SEC-06	Pertahanan prompt injection: instruksi di dalam pesan pengguna diperlakukan sebagai data; LLM tidak punya akses tool atau database
SEC-07	Rate limit per pengguna: 30 pesan/menit dan 500 panggilan LLM/hari
SEC-08	Log tidak memuat isi pesan mentah di level INFO; `parser_logs` menyimpan teks yang sudah disamarkan
SEC-09	Database dan volume terenkripsi at rest; backup harian dengan retensi 7 hari
SEC-10	/hapusakun menghapus semua data pengguna dalam 30 hari, termasuk dari backup yang dirotasi
SEC-11	Kebijakan privasi singkat ditampilkan saat onboarding
SEC-12	Dependensi dipindai (misalnya Trivy) di CI; image berjalan sebagai non-root dengan filesystem read-only

Pengujian dan kriteria penerimaan

MVP dianggap selesai bila seluruh kebutuhan P0 lolos kriteria di bawah dan dataset evaluasi parser mencapai target NFR-03.

Lapisan	Cakupan	Alat
Unit test	Parser aturan (nominal, waktu, kategori), validasi, kalkulasi budget	`go test`, tabel kasus
Evaluasi parser	≥ 300 kalimat berlabel; dijalankan setiap kali prompt atau aturan berubah; skor per jenis kesalahan	Skrip eval + laporan di CI
Contract test	Kompatibilitas protobuf antar-versi	`buf breaking`
Integration test	Service + Postgres + Redis sungguhan	Testcontainers
End-to-end	Alur chat lengkap di cluster kind dengan LLM di-mock	Skrip Telegram tiruan
Load test	50 pesan/detik selama 10 menit; HPA naik dan turun	k6 dengan plugin gRPC atau ghz
Chaos sederhana	Hapus Pod acak saat load test; tidak ada transaksi hilang atau ganda	`kubectl delete pod` terjadwal

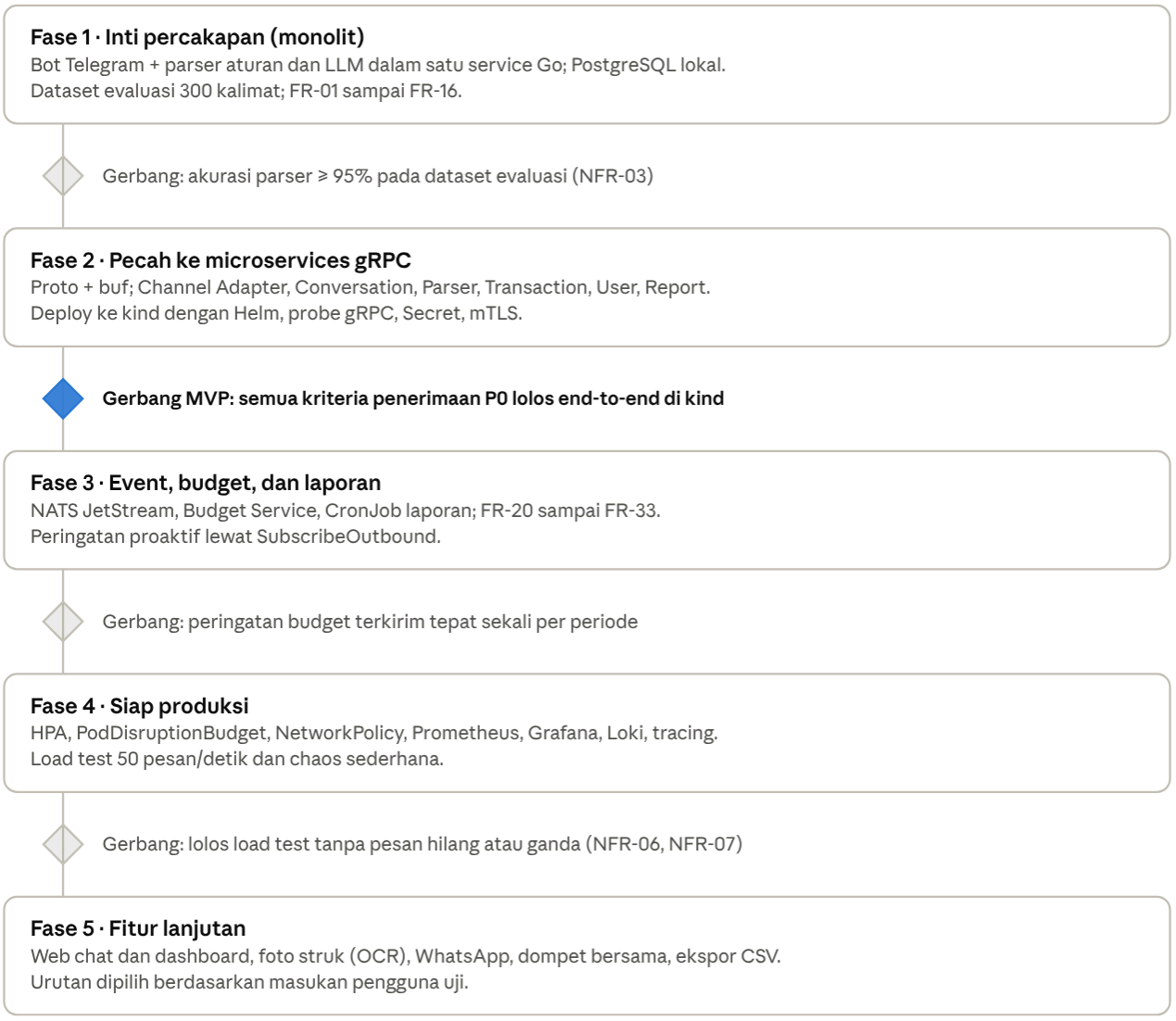
Kriteria penerimaan contoh:

- [ ] Pesan "bakso 15rb sama es teh goceng" menghasilkan tepat 2 transaksi dengan nominal Rp15.000 dan Rp5.000.
- [ ] Webhook yang sama dikirim dua kali hanya menghasilkan satu set transaksi.
- [ ] "yang bakso tadi 18rb" mengubah transaksi bakso terakhir dan mencatat jejak di `transaction_edits`.
- [ ] Saat LLM tidak merespons dalam 5 detik, pengguna menerima pertanyaan klarifikasi, bukan error.
- [ ] Pengguna A tidak dapat melihat atau mengubah transaksi pengguna B melalui RPC apa pun.
- [ ] Rolling update Parser Service selama load test tidak menyebabkan pesan hilang.
- [ ] Peringatan budget 80% terkirim tepat satu kali per periode.

Roadmap

Pengerjaan dibagi lima fase; setiap fase baru dimulai setelah gerbang sebelumnya terpenuhi, sehingga selalu ada versi yang berjalan.

MVP tercapai setelah Fase 2, saat semua kriteria P0 lolos di kind



roadmap Rupi · 5 fase, 4 gerbang

Fase 1 sengaja berupa monolit: kualitas parser adalah inti produk dan lebih mudah diasah sebelum sistem dipecah menjadi banyak service.

Risiko dan pertanyaan terbuka

Risiko	Dampak	Mitigasi
Parser salah membaca nominal (misalnya "15" dikira Rp15)	Data keuangan tidak akurat, pengguna kehilangan kepercayaan	Konfirmasi setiap transaksi, tombol Ubah, tanya balik saat ragu
Biaya API LLM membengkak	Proyek tidak berkelanjutan	Jalur aturan dulu, cache hasil, rate limit, pantau token per pengguna
Kompleksitas microservices berlebihan untuk tim kecil	Pengerjaan lambat, banyak waktu habis di infrastruktur	Mulai dari monolit modular di Fase 1, pecah bertahap
Cluster lokal kekurangan resource	Sulit menjalankan semua komponen + observability	Pakai limit kecil, nyalakan observability hanya saat dibutuhkan
Kebocoran data keuangan	Pelanggaran privasi	Bagian Keamanan, filter `user_id` di setiap query, minimalkan data ke LLM
Perubahan kebijakan API Telegram/WhatsApp	Kanal terganggu	Channel Adapter terpisah sehingga kanal bisa diganti

Pertanyaan terbuka:

- [ ] Penyedia LLM mana yang dipakai, dan apakah perlu model lokal sebagai cadangan?
- [ ] Di mana Rupi di-hosting untuk pengguna sungguhan: cloud managed Kubernetes atau VPS dengan k3s?
- [ ] Apakah mendukung beberapa dompet (tunai, rekening, e-wallet) sejak MVP atau mulai fase lanjutan?
- [ ] Bahasa pemrograman: Go untuk semua service, atau Python khusus untuk Parser Service?
- [ ] Siapa pengguna uji awal dan bagaimana mengumpulkan kalimat nyata untuk dataset evaluasi?